

Visão geral criada por IA



Yes, the absolute easiest native way to use it is by importing `node:readline/promises`, which allows you to use clean `async/await` syntax instead of clunky nested callbacks.  Node.js +1

If you prefer an even higher-level developer experience without configuring streams manually, you can also opt for popular **third-party libraries**. [↗](#)

Here is a breakdown of your best options, ranked from built-in fixes to external packages.

1. The Native Promises API (No installation required)

Node.js includes a built-in promise-based wrapper. It eliminates callback hell and allows you to capture user input sequentially using a simple `await` keyword.  Node.js +1

javascript

```
import * as readline from 'node:readline/promises';
import { stdin as input, stdout as output } from 'node:process';

const rl = readline.createInterface({ input, output });

// Clean, synchronous-looking asynchronous code
const name = await rl.question('What is your name? ');
const color = await rl.question('What is your favorite color? ');

console.log(`Hello ${name}! Your favorite color is ${color}.`);

rl.close();
```

Use o código com cuidado.



2. The `Inquirer.js` Library (Best for advanced CLI tools)

If your goal is to build an interactive command-line interface with menus, checkboxes, or input validation, you should skip `readline` entirely. The most popular community tool for this is `Inquirer`.

- **Why it is easier:** You declare your prompts as an array of configuration objects, and it handles all terminal formatting automatically.

Installation: `npm install inquirer`

- **Installation:** `npm install inquirer`

javascript

```
import inquirer from 'inquirer';

const answers = await inquirer.prompt([
  {
    type: 'input',
    name: 'username',
    message: 'Enter your username:',
  },
  {
    type: 'list',
    name: 'theme',
    message: 'Select a UI theme:',
    choices: ['Light', 'Dark', 'System'],
  }
]);

console.log(answers.username, answers.theme);
```

Use o código com cuidado.

3. The `readline-sync` Library (Best for simple scripts & beginners)

If you do not want to deal with `async`, `await`, or Promises at all, `readline-sync` halts script execution completely until the user presses enter.

- **Why it is easier:** It behaves exactly like Python's `input()` or Ruby's `gets`.
- **Installation:** `npm install readline-sync`

javascript

```
const readlineSync = require('readline-sync');

// No async wrapper function needed
const name = readlineSync.question('What is your name? ');
console.log(`Hi ${name}!`);
```

Use o código com cuidado.

Summary Comparison

Method	Syntax Type	Best Used For	Setup Effort
<code>node:readline/promises</code>	Promises (<code>async/await</code>)	Lightweight scripts without external dependencies.	Low (Built-in)
<code>Inquirer.js</code>	Promises / Objects	Professional CLI applications requiring complex inputs or validation.	Medium (Requires NPM install)
<code>readline-sync</code>	Synchronous	Quick scripts, prototyping, or educational exercises.	Low (Requires NPM install)

Would you like help rewriting an **existing script** using one of these methods, or are you trying to build a **specific feature** like a multi-choice menu?

 6 sites

Node.js Readline Module - W3Schools

Note: The Readline module is built into Node. js, so no additional installation is required. It's perfect for any application that...

 W3Schools

Readline | Node.js v26.5.1 Documentation

The following simple example illustrates the basic use of the node:readline module. import * as readline from 'node:readline/promi...

 Node.js

Readline in Node.js — Beginner Series | by Farhad | Medium

17 de jun. de 2025 — The readline module has two interfaces: * **Promise-based API**
Provides a cleaner way to handle asynchronous operations using asy...

 Medium



Mostrar tudo

Pergunte o que quiser